

Here's the full implementation flow in order, broken into phases you can follow one by one.

Phase 1 — Dev environment & project foundation

1. Install Java 21, IntelliJ IDEA, Docker Desktop, and Postman
 2. Learn the Spring Boot project structure — understand how controllers, services, repositories, and models are layered
 3. Go to start.spring.io and generate the Patient Service project with dependencies: Spring Web, Spring Data JPA, H2, Validation, Lombok
 4. Verify the app runs on localhost with a basic health check
-

Phase 2 — Patient Service core (in-memory)

5. Create the `Patient` entity class with fields: id, name, email, address, dateOfBirth
 6. Create `PatientRepository` extending `JpaRepository` — this gives you all DB operations for free
 7. Create `PatientService` — business logic layer, injected with the repository
 8. Create `PatientController` — REST endpoints, injected with the service
 9. Implement `GET /patients` and `GET /patients/{id}` — test both with Postman
 10. Create `PatientRequestDTO` — a separate object for incoming request data, decoupled from the entity
 11. Implement `POST /patients` using the DTO — test patient creation
 12. Add Bean Validation to the DTO — `@NotBlank`, `@Email`, `@NotNull` on each field
 13. Create a global `@ControllerAdvice` exception handler to return clean JSON error responses for validation failures
 14. Add custom exception classes for business logic errors (e.g. `PatientNotFoundException`, `EmailAlreadyExistsException`)
 15. Implement `PUT /patients/{id}` — update patient, with its own validation and duplicate email check
 16. Implement `DELETE /patients/{id}` — remove patient, throw exception if not found
 17. Add Springdoc OpenAPI dependency — auto-generates `/swagger-ui.html` API docs
-

Phase 3 — Real database with Docker

18. Learn Docker basics — images, containers, volumes, and docker-compose
19. Replace H2 with PostgreSQL — add the PostgreSQL driver dependency, remove H2

20. Write a `docker-compose.yml` that starts a Postgres container with environment variables for DB name, user, and password
 21. Configure `application.yml` in Patient Service with the Postgres connection string
 22. Add a health check in docker-compose so the service waits for Postgres to be ready before starting
 23. Write a `Dockerfile` for Patient Service — base image, copy jar, expose port, entrypoint
 24. Build and run the full stack with `docker compose up` — verify end to end with Postman
-

Phase 4 — gRPC & Billing Service

25. Learn gRPC concepts — protocol buffers, service definitions, generated stubs, sync RPC
 26. Create the Billing Service as a new Spring Boot project with gRPC dependencies
 27. Write a `billing.proto` file defining the `BillingService` RPC method and request/response message types
 28. Configure the `protobuf-maven-plugin` to generate Java stubs from the proto file
 29. Implement the gRPC server in Billing Service — extend the generated base class, override the RPC method, save a billing account
 30. Test the gRPC server in isolation using a gRPC client tool (e.g. `grpcurl` or `BloomRPC`)
 31. Write a `Dockerfile` for Billing Service and add it to `docker-compose.yml`
 32. Add the gRPC client dependency to Patient Service
 33. Create a `BillingServiceGrpcClient` class in Patient Service — configure the channel to point at the Billing Service container
 34. Call the gRPC client from within the `createPatient` method in `PatientService` — so every new patient triggers a billing account creation
 35. Test the full flow: create patient via REST → gRPC call fires → billing account created
-

Phase 5 — Kafka event streaming

36. Learn Kafka concepts — brokers, topics, producers, consumers, consumer groups, offsets
37. Add a Kafka broker to `docker-compose.yml` using the Confluent or Bitnami Kafka image
38. Add Spring Kafka dependency to Patient Service
39. Create a `KafkaProducerConfig` class in Patient Service — configure serialiser, broker address

40. Define a `patient_event.proto` file for the Kafka event message schema — `PatientCreatedEvent` with patient fields
 41. Generate Java stubs from the event proto
 42. Create a `KafkaProducer` class in Patient Service — takes a `PatientCreatedEvent` and publishes it to the `patient-events` topic
 43. Call the Kafka producer from the `createPatient` method after saving the patient and calling billing
 44. Configure Kafka topic settings in `application.yml` — topic name, partition count
 45. Test: create patient → check Kafka logs to confirm the event was published
 46. Create the Analytics Service as a new Spring Boot project with Spring Kafka dependency
 47. Copy the proto file to Analytics Service and generate stubs
 48. Create a `KafkaConsumerConfig` class — configure deserialiser, broker address, consumer group ID
 49. Create a `PatientEventConsumer` class annotated with `@KafkaListener` — processes incoming events
 50. Write a `Dockerfile` for Analytics Service and add it to `docker-compose.yml`
 51. Test the full async flow: create patient → Kafka event → analytics consumer logs the event
-

Phase 6 — Notification Service (add-on)

52. Create Notification Service as a new Spring Boot project
 53. Add Spring Kafka dependency and configure a consumer in the same `patient-events` topic but with a different consumer group ID
 54. On receiving a `PatientCreatedEvent`, call an email provider SDK (e.g. `JavaMailSender` or `SendGrid`) to send a welcome email
 55. Write a `Dockerfile` and add to `docker-compose.yml`
 56. Test: create patient → two consumers fire independently — analytics logs, notification sends email
-

Phase 7 — Audit Service (add-on)

57. Create Audit Service as a new Spring Boot project with Spring Kafka and Spring Data JPA
58. Create an `AuditLog` entity — `eventType`, `payload`, `actorId`, `timestamp` — with no update or delete operations allowed
59. Subscribe to Kafka events and write every event to the audit table as an immutable record

- 60. Expose a read-only `GET /audit-logs` REST endpoint for admin queries
 - 61. Write a `Dockerfile` and add to `docker-compose.yml`
-

Phase 8 — API Gateway

- 62. Create the API Gateway as a new Spring Boot project with Spring Cloud Gateway dependency
 - 63. Define routing rules in `application.yml` — map URL path prefixes to downstream service hostnames
 - 64. Write a `Dockerfile` and add to `docker-compose.yml`
 - 65. Test: all requests go through the gateway port and are correctly forwarded
-

Phase 9 — Auth Service

- 66. Create Auth Service as a new Spring Boot project with Spring Security, Spring Data JPA, PostgreSQL, and jjwt dependencies
 - 67. Create a `users` table with columns: id, email, password (hashed), role
 - 68. Write a SQL seed file that inserts an initial admin user with a BCrypt-hashed password
 - 69. Configure the Auth Service `Dockerfile` to run the seed SQL on first start
 - 70. Create `LoginRequestDTO` (email + password) and `LoginResponseDTO` (token)
 - 71. Create `AuthController` with `POST /auth/login` — receives credentials, validates, returns JWT
 - 72. Create `UserRepository` to look up users by email
 - 73. Configure BCrypt `PasswordEncoder` as a Spring bean
 - 74. Create `JwtUtil` class — `generateToken(userId, role)` and `validateToken(token)` methods using the jjwt library
 - 75. Wire everything together in `AuthService` — find user, check password, generate and return token
 - 76. Test the login endpoint with Postman — valid credentials return a JWT, invalid return 401
 - 77. Implement `POST /auth/validate` — accepts a token, verifies signature and expiry, returns claims
 - 78. Test the validate endpoint with a valid token and an expired/tampered token
 - 79. Write a `Dockerfile` for Auth Service and add to `docker-compose.yml`
-

Phase 10 — JWT filter in the Gateway

80. Add a `JwtAuthenticationFilter` class in the API Gateway — a `GatewayFilter` that intercepts every request
 81. The filter extracts the `Authorization` header, calls `POST /auth/validate` on the Auth Service, and either forwards the request or returns 401
 82. Register the filter in `application.yml` and apply it to all routes except `/auth/login`
 83. Test: unauthenticated request to `/api/patients` returns 401, authenticated request succeeds
 84. Add global exception handling in the gateway for auth failures — return clean JSON instead of a stack trace
 85. Expose Auth Service OpenAPI docs through the gateway routing rules
-

Phase 11 — Integration testing

86. Create a dedicated integration-test module as a separate Maven/Gradle project
 87. Add Testcontainers dependency — it spins up real Docker containers (Postgres, Kafka) during test runs
 88. Add Spring Boot Test and RestAssured or MockMvc for HTTP assertions
 89. Write a test for `POST /auth/login` with valid credentials — assert 200 and a token in the response
 90. Write a test for `POST /auth/login` with invalid credentials — assert 401
 91. Write a test for `GET /patients` with a valid JWT — assert 200 and a list of patients
 92. Write a test for `GET /patients` with no token — assert 401 from the gateway filter
 93. Run all tests and confirm they pass against the real containerised stack
-

Phase 12 — Kubernetes infrastructure

94. Install kubectl, Helm, and either minikube (local) or eksctl (AWS EKS)
95. Install Terraform for infrastructure provisioning
96. Write Terraform scripts to provision: VPC, subnets, EKS cluster, RDS (Postgres) instances for each service, MSK (Kafka) cluster
97. Apply Terraform locally against LocalStack first to validate, then against real AWS
98. For each service, write Kubernetes manifests: `Deployment`, `Service` (ClusterIP), `ConfigMap`, `Secret`
99. Configure liveness and readiness probes in each Deployment pointing at `/actuator/health`
100. Write `HorizontalPodAutoscaler` for Patient Service and Auth Service
101. Install the NGINX Ingress Controller in the cluster
102. Write an `Ingress` resource defining path-based routing rules to each service

103. Install the External Secrets Operator and configure it to pull DB passwords and JWT secret from AWS Secrets Manager
 104. Deploy all manifests with `kubectl apply -f k8s/` and verify pods are running
 105. Test the full stack end to end through the Ingress hostname
-

Phase 13 — CI/CD pipeline

106. Push all service code to a GitHub monorepo with one folder per service
 107. Write a GitHub Actions workflow for each service — triggers on push to `main`
 108. Each workflow: run unit tests → run integration tests → build Docker image → push to AWS ECR
 109. Add a deploy step: update the Kubernetes Deployment image tag using `kubectl set image` or `helm upgrade`
 110. Install ArgoCD in the cluster — configure it to watch the `k8s/` folder in your repo and auto-sync on merge to `main`
 111. Add environment promotion: a `staging` namespace and a `production` namespace, promoted by a manual approval step in GitHub Actions
 112. Add `cfn-lint` and `checkov` steps in CI to scan Terraform/Kubernetes manifests for security issues before any deploy
-

Phase 14 — Observability

113. Add Micrometer and Spring Boot Actuator to every service — exposes `/actuator/metrics` and `/actuator/health`
114. Deploy Prometheus in the cluster — configure it to scrape each service's actuator endpoint
115. Deploy Grafana — connect it to Prometheus as a data source
116. Build dashboards in Grafana: request rate, error rate, latency (p50/p95/p99) per service
117. Add Micrometer Tracing with Zipkin or Tempo — traces requests across gateway → patient service → billing → kafka → analytics
118. Deploy Loki for log aggregation — configure each service to ship logs to Loki, query them in Grafana
119. Set up Grafana alerting — alert on error rate above 1% or any pod going down, routed to Slack or PagerDuty